

Abdelghafour Rahmouni : 20264224

Marc Olivier Jean Paul : 20241763

Devoir 2 - IFT 2125

1- Trouvez l'erreur

Question 1

La preuve est fautive.

On peut le voir avec ce contre exemple:

Soit $f(n) = 2n$ et $g(n) = n$, on a que

$f(n) > g(n) \quad \forall n \geq n_0 = 1$. Alors:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{2n}{n} = 2 \neq +\infty$$

Question 2

L'exemple est bon, mais pas l'énoncé.

On peut le voir avec ce contre exemple:

$$f(n) = \frac{n}{10}, g(n) = \sqrt{n}, h(n) = \sqrt{n}$$

On a : $\frac{n}{10} \geq \sqrt{n} \quad \forall n \geq n_0 = 100$

Avec l'exemple, on sait déjà que $\frac{n}{10} \notin O(\sqrt{n})$

donc $f(n) \notin O(h(n))$ or, $\sqrt{n} \leq c \cdot \sqrt{n}$

En utilisant la définition:

$g(n) \in O(h(n)) \Rightarrow (\exists c \in \mathbb{R}^+) (\exists n_0 \in \mathbb{N})$ tel que $\forall n \geq n_0, g(n) \leq c \cdot h(n)$

$$\sqrt{n} \leq c \cdot \sqrt{n}$$

Si on prend $c=1$, alors $\sqrt{n} \leq \sqrt{n} \quad \forall n \geq n_0 = 0$

Donc $g(n) \in O(h(n))$

Question 3

Il y a une erreur dans la récurrence:

Au début, on a:

$$t_n - 4t_{n-1} + 4t_{n-2} = 4(n+1)4^n (\Delta)$$

On multiplie par 4:

$$4t_n - 16t_{n-1} + 16t_{n-2} = 4(n+1)4^{n+1}$$

On remplace n par $n-1$:

$$4t_{n-1} - 16t_{n-2} + 16t_{n-3} = 4n4^n (\square) \quad \text{L'erreur était ici}$$

On soustrait $\Delta - \square$:

$$t_n - 4t_{n-1} + 4t_{n-2} = 4(n+1)4^n$$

$$- \quad 4t_{n-1} - 16t_{n-2} + 16t_{n-3} = 4n4^n$$

$$t_n - 8t_{n-1} + 20t_{n-2} - 16t_{n-3} = 4 \cdot 4^n (*)$$

La récurrence n'est toujours pas homogène

On multiplie par 4:

$$4t_n - 32t_{n-1} + 80t_{n-2} - 64t_{n-3} = 16 \cdot 4^n$$

On remplace n par $n-1$:

$$4t_{n-1} - 32t_{n-2} + 80t_{n-3} - 64t_{n-4} = 16 \cdot 4^{n-1} \\ = 4 \cdot 4^n (0)$$

On soustrait $* - 0$:

$$t_n - 8t_{n-1} + 20t_{n-2} - 16t_{n-3} = 4 \cdot 4^n$$

$$- \quad 4t_{n-1} - 32t_{n-2} + 80t_{n-3} - 64t_{n-4} = 4 \cdot 4^n$$

$$t_n - 12t_{n-1} + 52t_{n-2} - 96t_{n-3} + 64t_{n-4} = 0$$

La nous avons une récurrence homogène.

Question 4

Il y a une erreur dans la justification de la preuve:

On a: $\lim_{n \rightarrow +\infty} \frac{2^n}{4^n} = \frac{1}{2}$. Jusque là, il n'y a pas d'erreur.

Mais, selon le critère de la limite, cela signifie que $2^n \in O(4^n)$ et que $4^n \in O(2^n)$.

Donc $2^n \notin \Omega(4^n)$

Question 5

Il y a une erreur dans la preuve:

$2^n \in \Omega(4^n) \Rightarrow (\exists c \in \mathbb{R}^+) (\exists m_0 \in \mathbb{N}) \text{ tel que } \forall n \geq m_0, 2^n \geq c \cdot 4^n$

Dans la justification, nous avons $c = \frac{1}{2}$ et $m_0 = 1$

Donc: $\left(\frac{1}{2}\right)^n \geq \frac{1}{2} \quad \forall n \geq 1$

On remarque que c'est vrai pour $n = m_0: \left(\frac{1}{2}\right)^1 \geq \frac{1}{2} \Rightarrow \frac{1}{2} = \frac{1}{2}$

mais si $n = 2$:

$\left(\frac{1}{2}\right)^2 \geq \frac{1}{2} \Rightarrow \frac{1}{4} \geq \frac{1}{2}$ ce qui est faux!

Donc $2^n \notin \Omega(4^n)$

2- Diviser pour régner

Informations de l'ordi:

OS: Windows 11 Home

Mémoire: 16.0 GB RAM

CPU: Intel(R) Core(TM) i7-11800H

Pour trouver le seuil optimal, nous avons décidé de faire une fonction "seuil_optimal" qui va permettre de trouver le seuil optimal qu'il faut.

```
def seuil_optimal(array):  
    """  
    Fonction auxiliaire pour déterminer le seuil optimal pour le tri hybride  
    à partir du array donné.  
    """  
    seuil_opt = 0  
    meilleur_temps = float('inf')  
  
    for seuil in range((int)(len(array) * 1.5)):  
        copie = array.copy()  
        debut = time.perf_counter()  
        tri_hybride(copie, seuil)  
        temps_ecoule = time.perf_counter() - debut  
  
        if temps_ecoule < meilleur_temps:  
            meilleur_temps = temps_ecoule  
            seuil_opt = seuil  
  
    return seuil_opt
```

Vu qu'on ne sait pas quel tableau sera passé en paramètre, nous nous sommes dit qu'il valait donc mieux chercher le seuil par rapport à la longueur du tableau.

Ainsi, on s'assure d'avoir le seuil optimal pour utiliser la bonne fonction de tri.

6 - Génération de paysage en 3D :

Le premier algorithme "generate_heightmap" génère une carte des hauteurs en créant une base circulaire décroissante autour d'un centre, à laquelle il ajoute 50 bosses aléatoires pour créer du relief, puis applique 10 passes de lissage (moyenne 3×3) pour adoucir la surface.

Le second algorithme "generate_island_terrain" parcourt cette carte et, pour chaque point ayant une hauteur significative, génère un cube coloré en fonction de l'altitude (plage ou végétation), en ajoutant aléatoirement des arbres (tronc + feuillage) dans les zones de hauteur intermédiaire pour enrichir visuellement l'île.